COURSE CODE
SBT-DF202

COURSE TITLE
Computer and Digital Forensics

LAB TITLE
Data Carving with XXD, Binwalk, and Scalpel

NAME
Kumpon Emmanuel Nandom

STUDENT ID
2025/FWSD/11325

DATE
05/09/2026

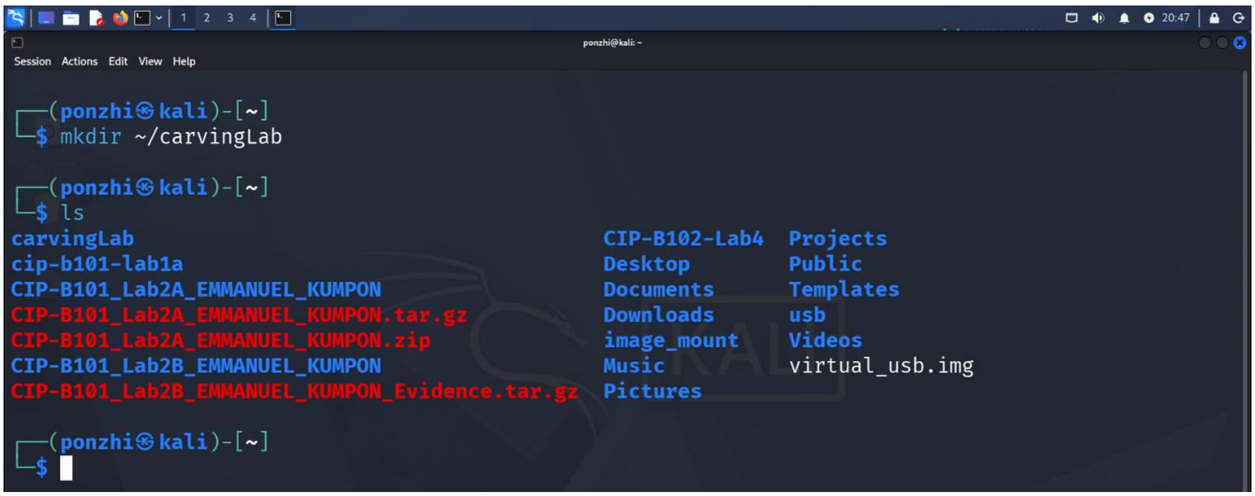
PLATFORM USED
Kali Linux

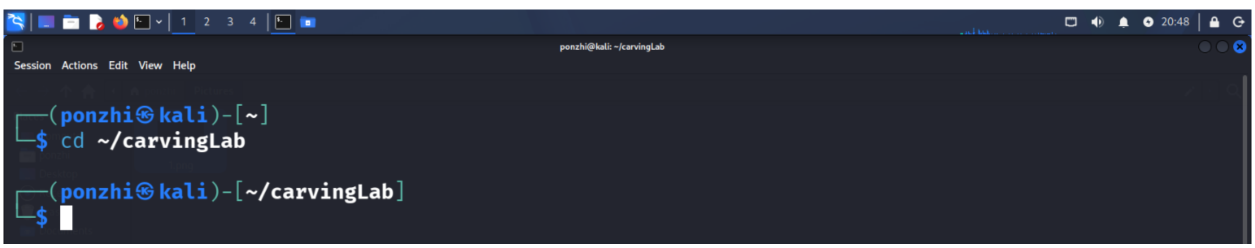
INSTRUCTOR
Mr. Aminu Idris, AMCPN
Commandant

OBJECTIVES

By completing this lab, I have demonstrate the ability to explain why data carving is required when file-system metadata is missing or unreliable, using xxd to examine binary file content and identify headers and footers, recognize JPEG SOI and EOI signatures, reconstruct binary content from a hexadecimal dump, using the Sleuth Kit to examine inode, block and sector-level data, identify and extract embedded content using Binwalk; recover deleted files from a forensic USB image using Scalpel, verify recovered evidence using cryptographic hashes; and prepare a professional forensic report containing commands, screenshots, findings and limitations.

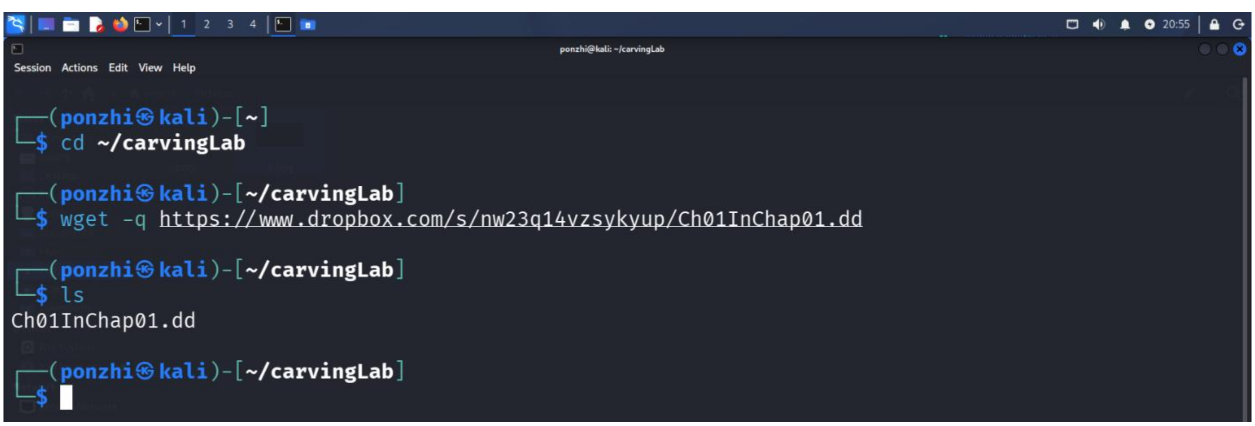
Creating a dedicated working directory



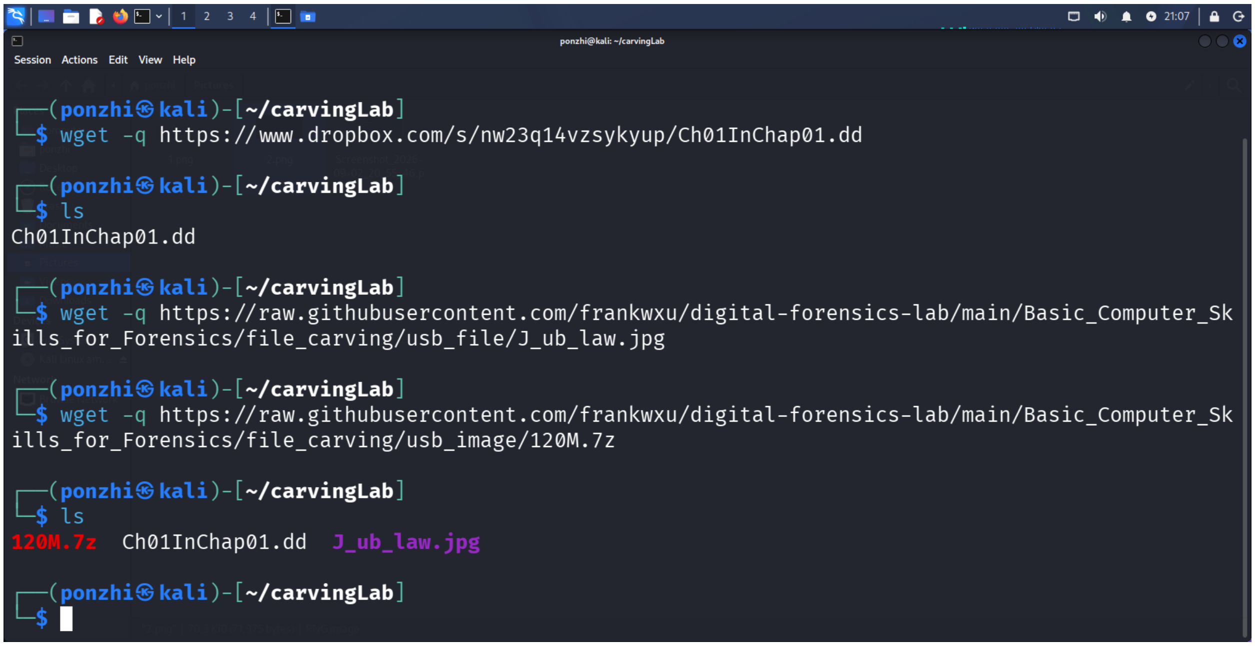


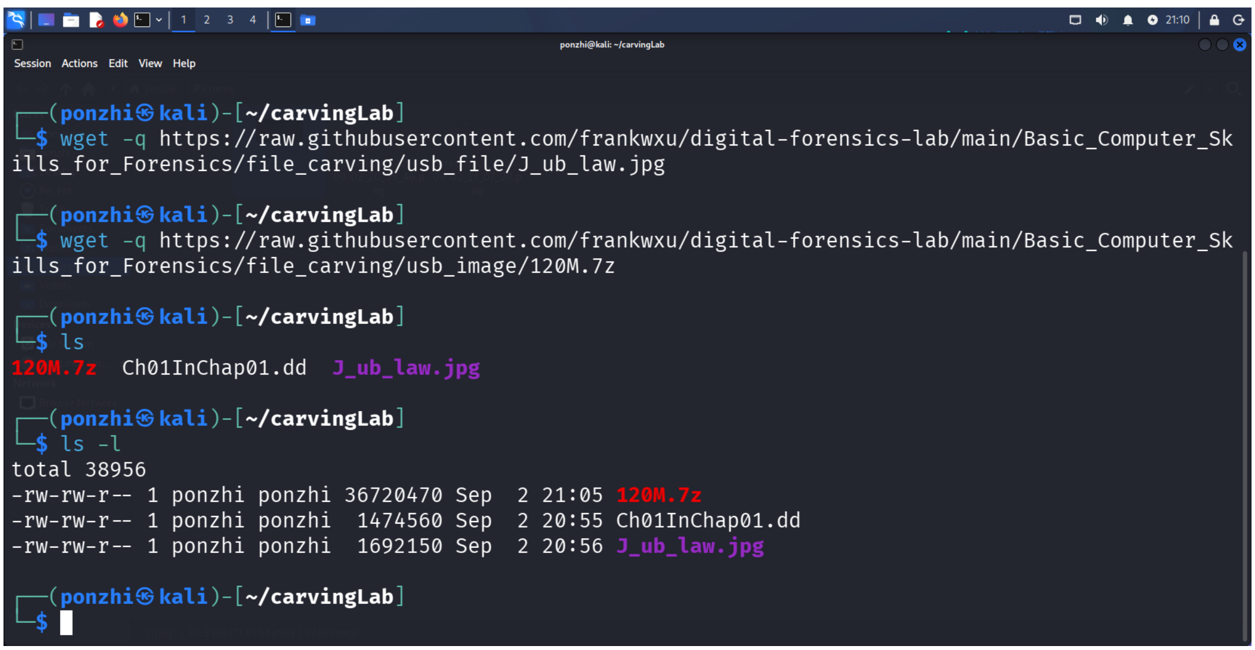
Downloading evidence and hashes

Ch01InChap01.dd



J_ub_law.jpg and 120M.7z





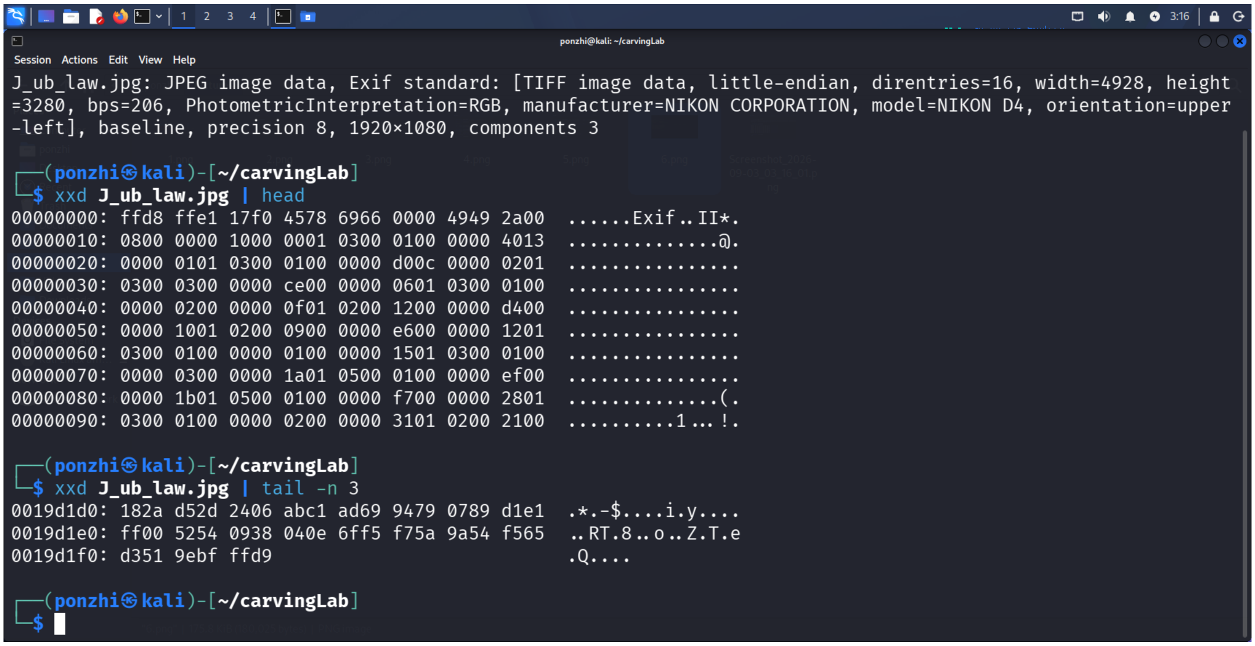
Hashes

```
ponzhi@kali: ~/carvingLab
Session  Actions  Edit  View  Help

┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ hashdeep -c md5,sha256 Ch01InChap01.dd J_ub_law.jpg 120M.7z | tee evidence_hashes_original.txt
%%%% HASHDEEP-1.0
%%%% size,md5,sha256,filename
## Invoked from: /home/ponzhi/carvingLab
## $ hashdeep -c md5,sha256 Ch01InChap01.dd J_ub_law.jpg 120M.7z
##
1474560,a117773bcf1fc88ec0ab8e0a349fbbcb,3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122,/home/
ponzhi/carvingLab/Ch01InChap01.dd
1692150,83a360ac7f7e0ca318e5bfe39f95f137,238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468,/home/
ponzhi/carvingLab/J_ub_law.jpg
36720470,dfe7b5424e54cd1bf50d5df47aceeb3c,2d2a3d93c9ec65bcad9f89c9894429ea72caa8add07726a3b96ec9a6ab6a58ce,/home
/ponzhi/carvingLab/120M.7z

┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ 
```

Part A - File Signatures and Hex Analysis

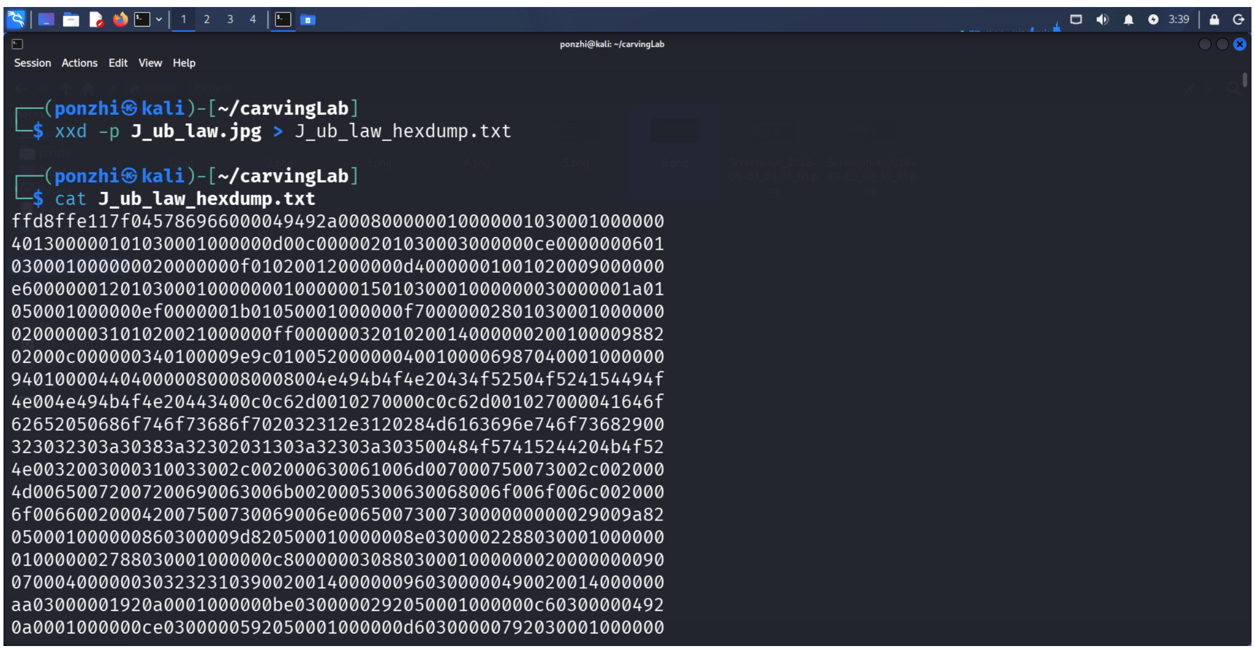
Confirm the file type and look at the header/tail with xxd

```
ponzhi@kali: ~/carvingLab
Session  Actions  Edit  View  Help

┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ file J_ub_law.jpg
J_ub_law.jpg: JPEG image data, Exif standard: [TIFF image data, little-endian, direntries=16, width=4928, height
=3280, bps=206, PhotometricInterpretation=RGB, manufacturer=NIKON CORPORATION, model=NIKON D4, orientation=upper
-left], baseline, precision 8, 1920x1080, components 3

┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ xxd J_ub_law.jpg | head
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00  ......Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013  ..............@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201  ................
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100  ................
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400  ................
00000050: 0000 1001 0200 0900 0000 e600 0000 1201  ................
00000060: 0300 0100 0000 0100 0000 1501 0300 0100  ................
00000070: 0000 0300 0000 1a01 0500 0100 0000 ef00  ................
00000080: 0000 1b01 0500 0100 0000 f700 0000 2801  ..............(.
00000090: 0300 0100 0000 0200 0000 3101 0200 2100  ..........1...!.

┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ 
```



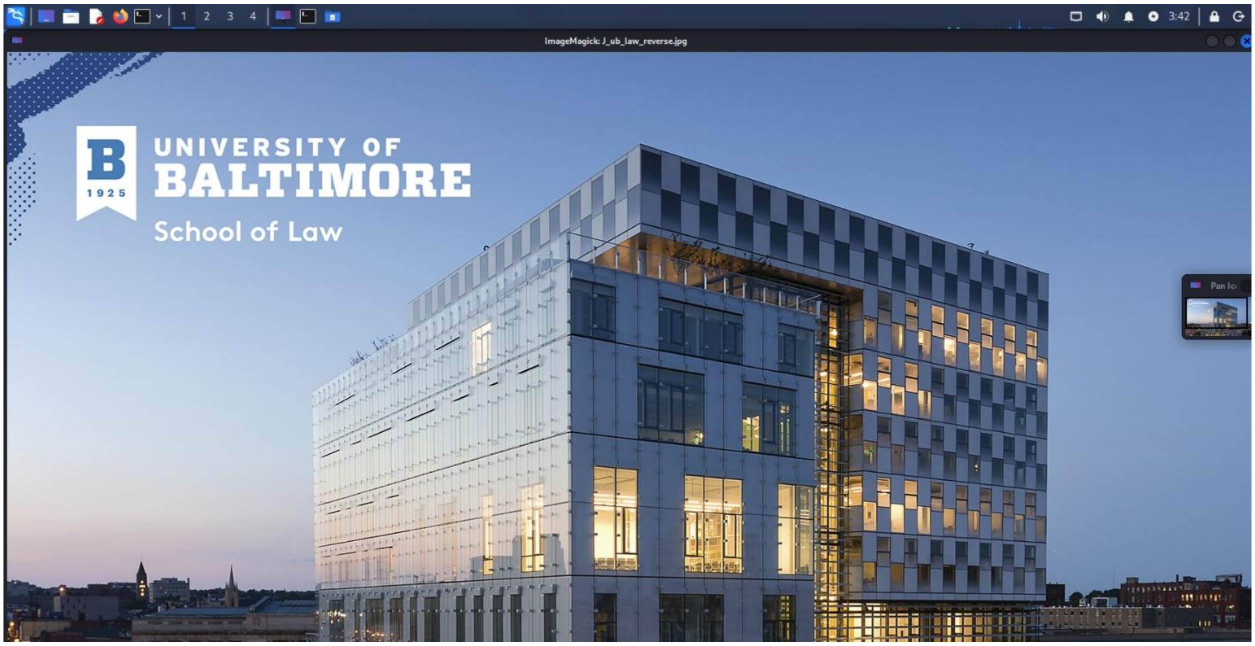
JPEG files begin with ffd8 (SOI) and end with ffd9 (EOI).

Create a plain (unformatted) hex dump and reconstruct the file from it



Convert a plain hexdump to back into binary

Image recovered

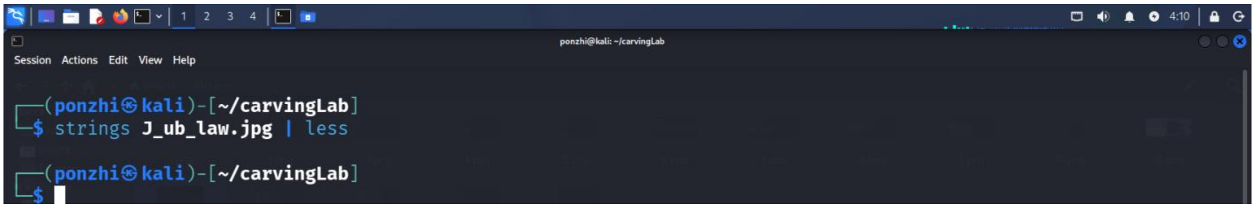


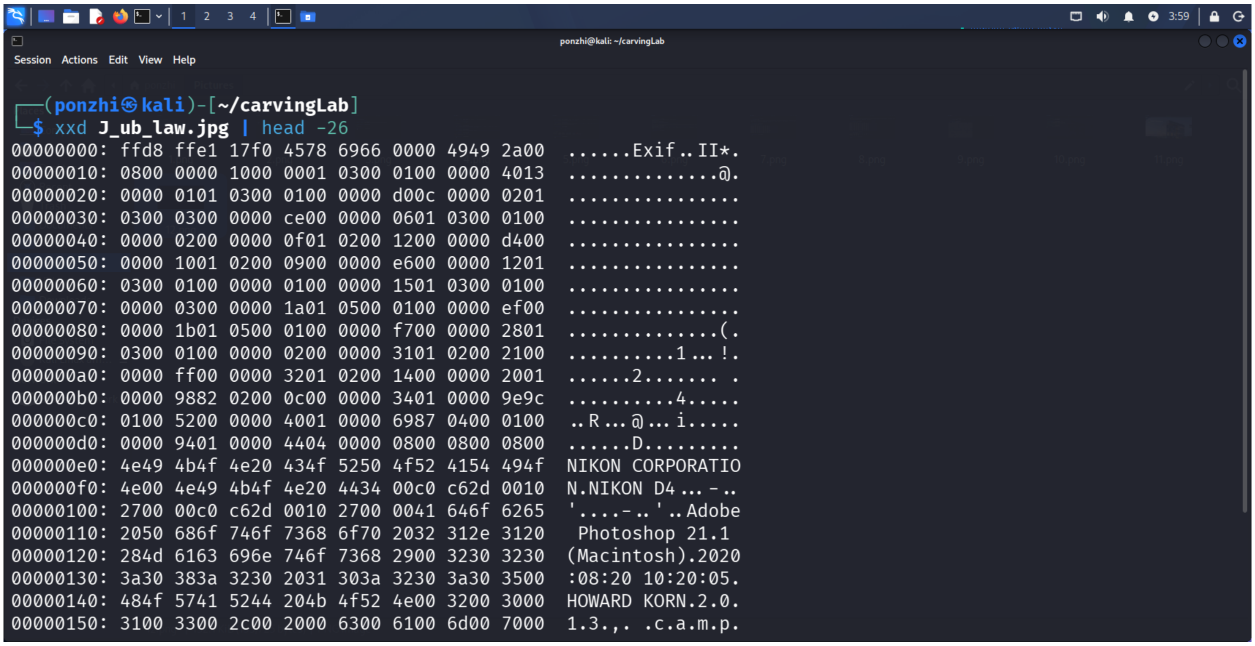
Verify the reconstruction with hashes

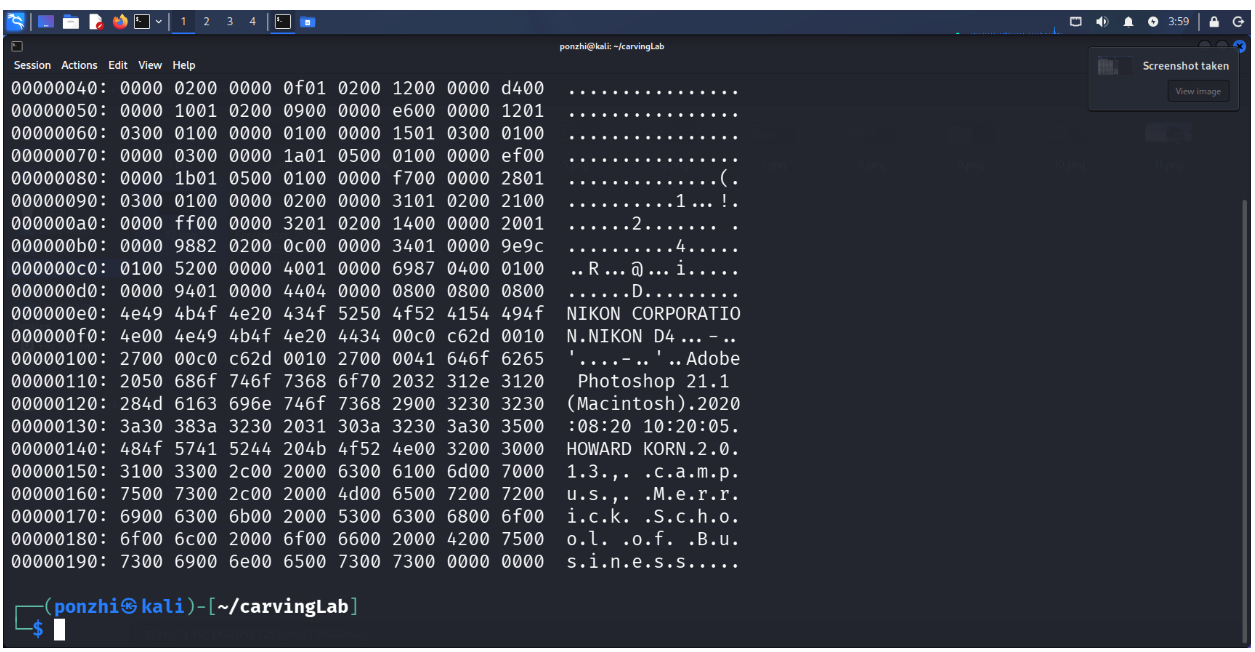
The hashes for the original and the reconstructed file match exactly.

Part B - Examine Embedded Metadata-Like Content

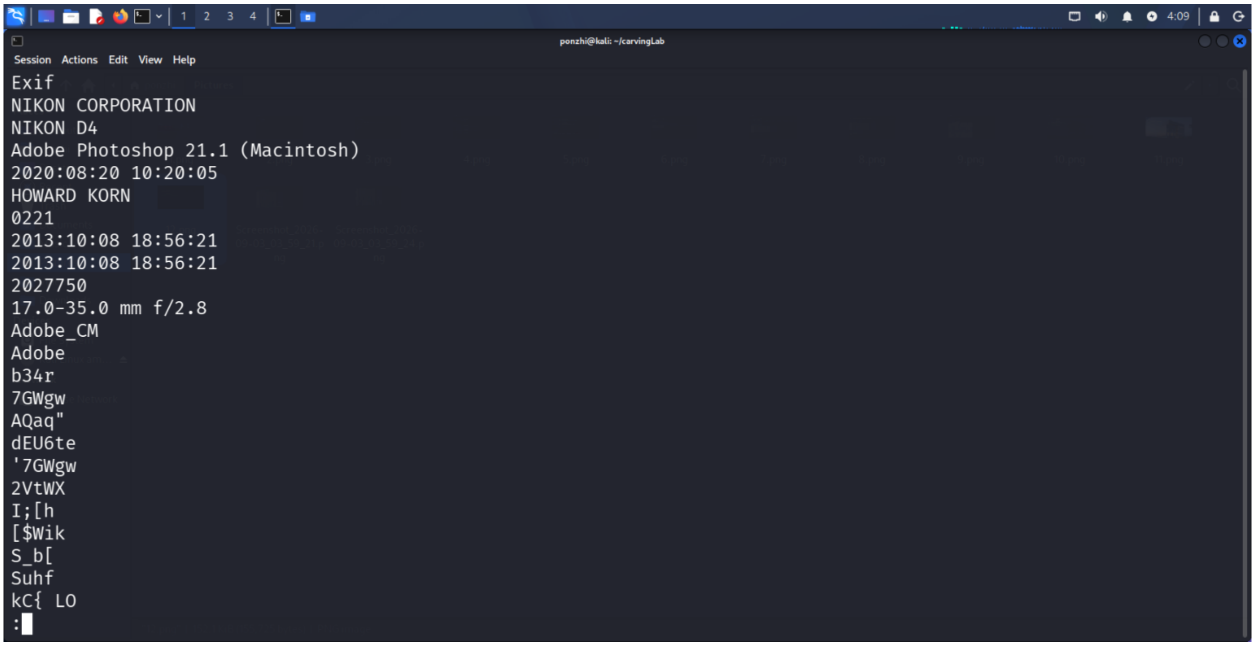
Broad text scan

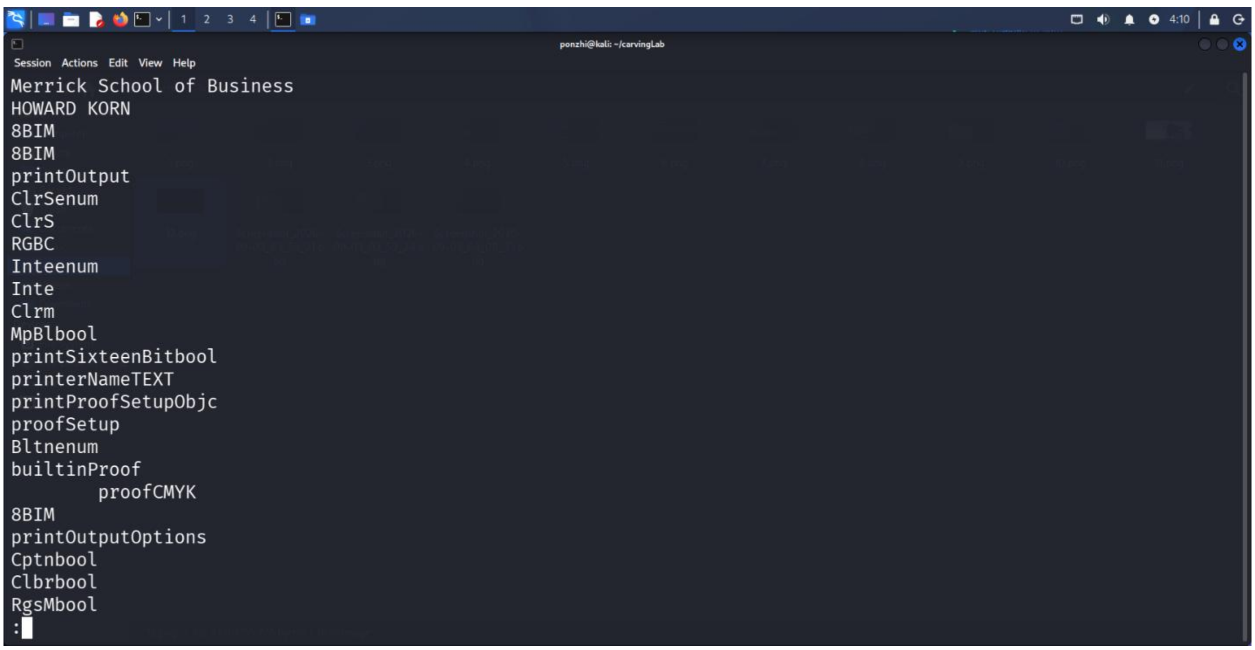




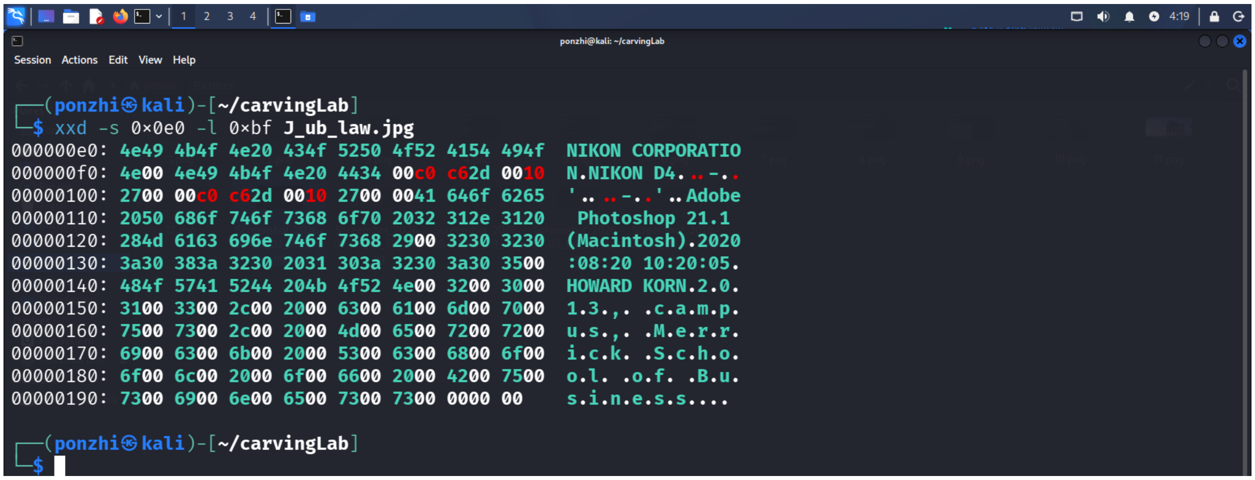


Look through this for camera make/model, software, dates, or names





Hexdump Photoshop info starts from 0x0e0 and list 0xbf bytes

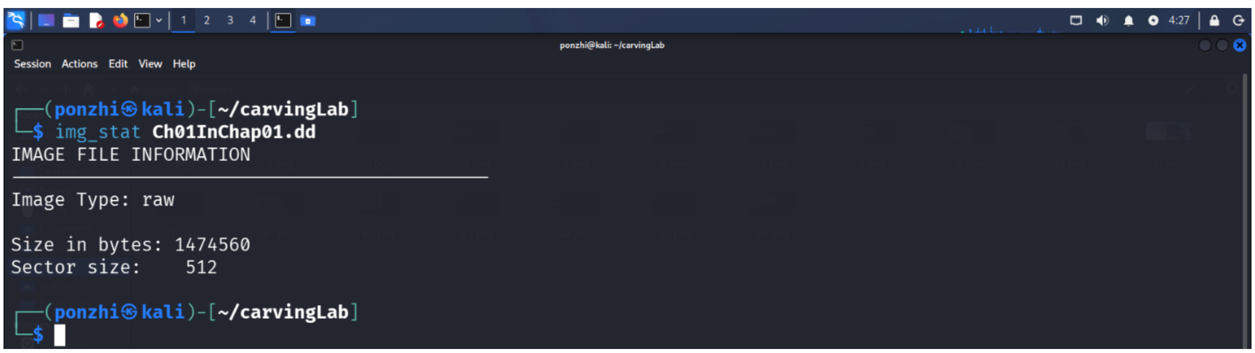


Part C - Direct Data-Unit Examination with The Sleuth Kit

Using Ch01InChap01.dd

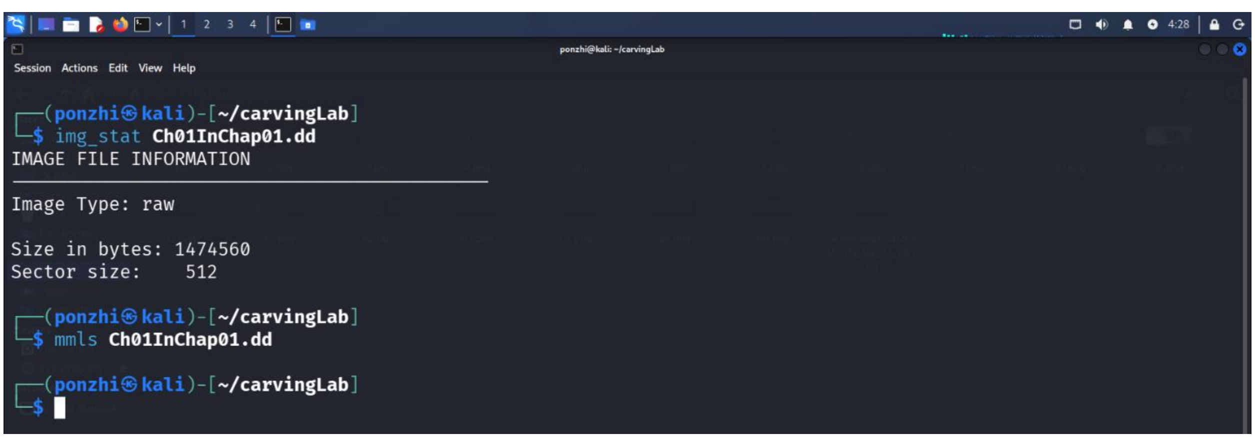
Basic image info

img_stat Ch01InChap01.dd



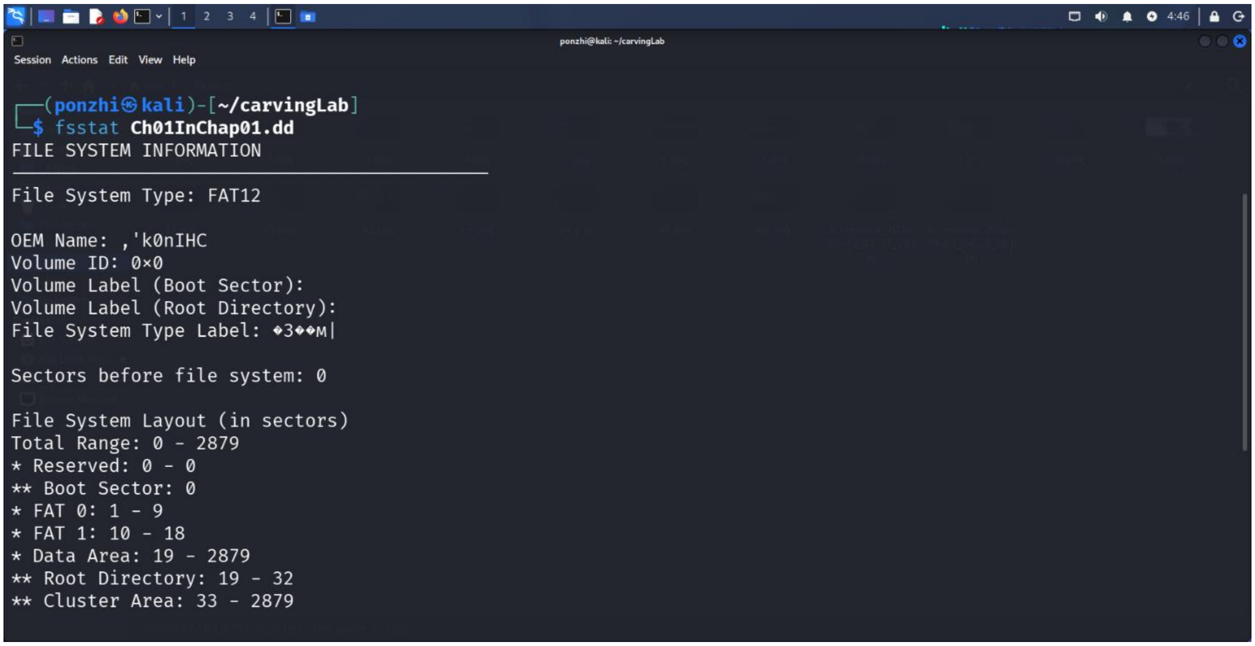
Partition layout needed before file-system-level commands, in case the image has multiple partitions

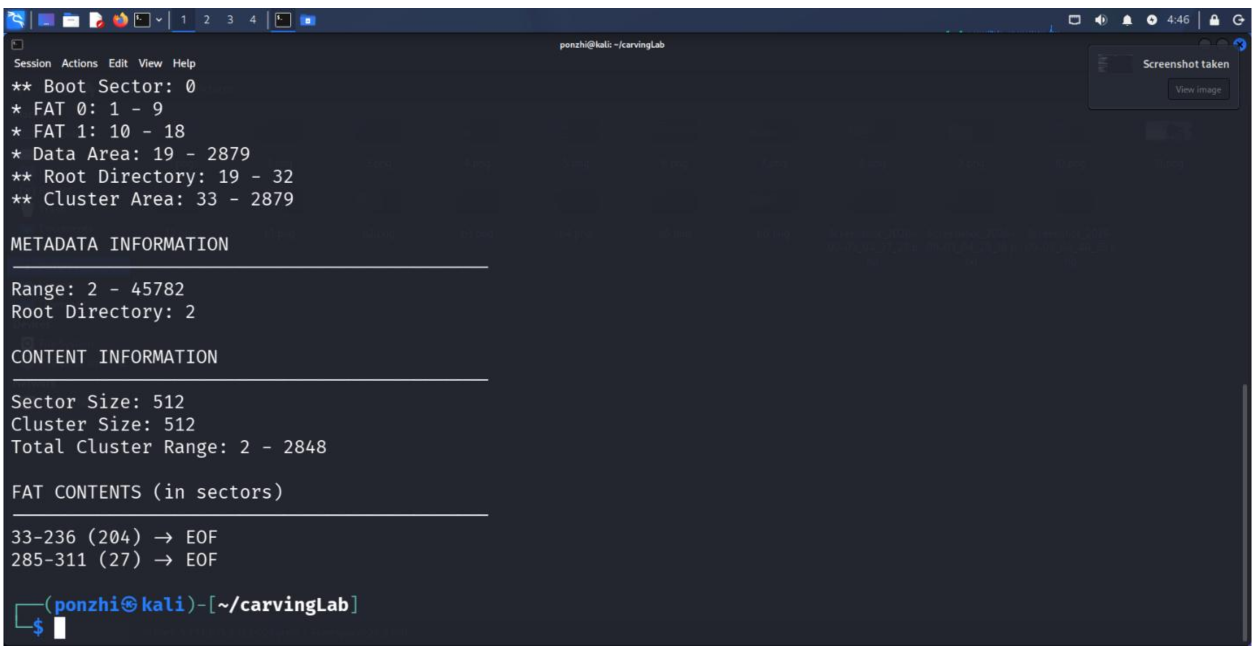
mmls Ch01InChap01.dd



File-system details use the correct offset from mmls if the image isn't partition-less; omit -o if img_stat/mmls shows a single filesystem starting at sector 0

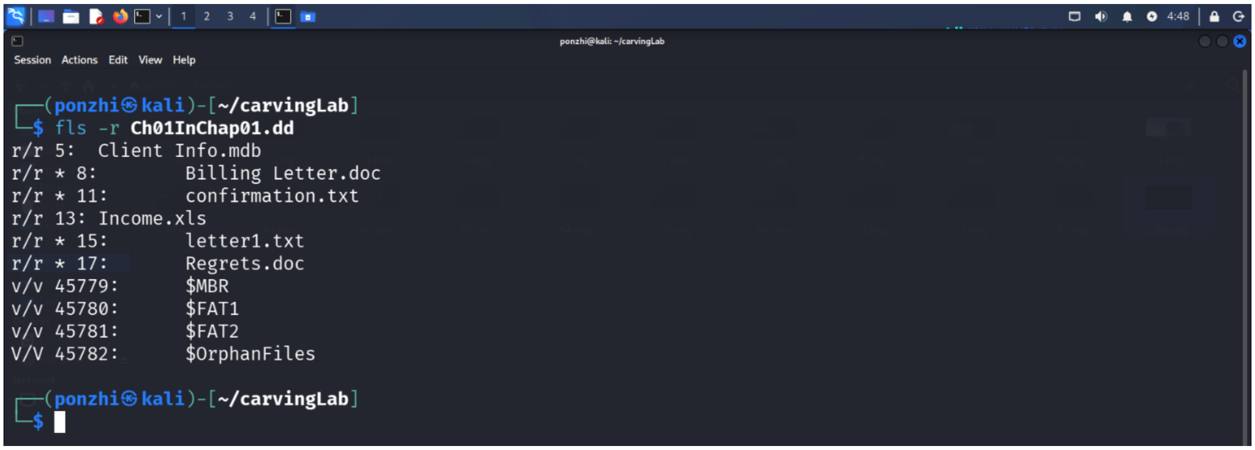
fsstat Ch01InChap01.dd





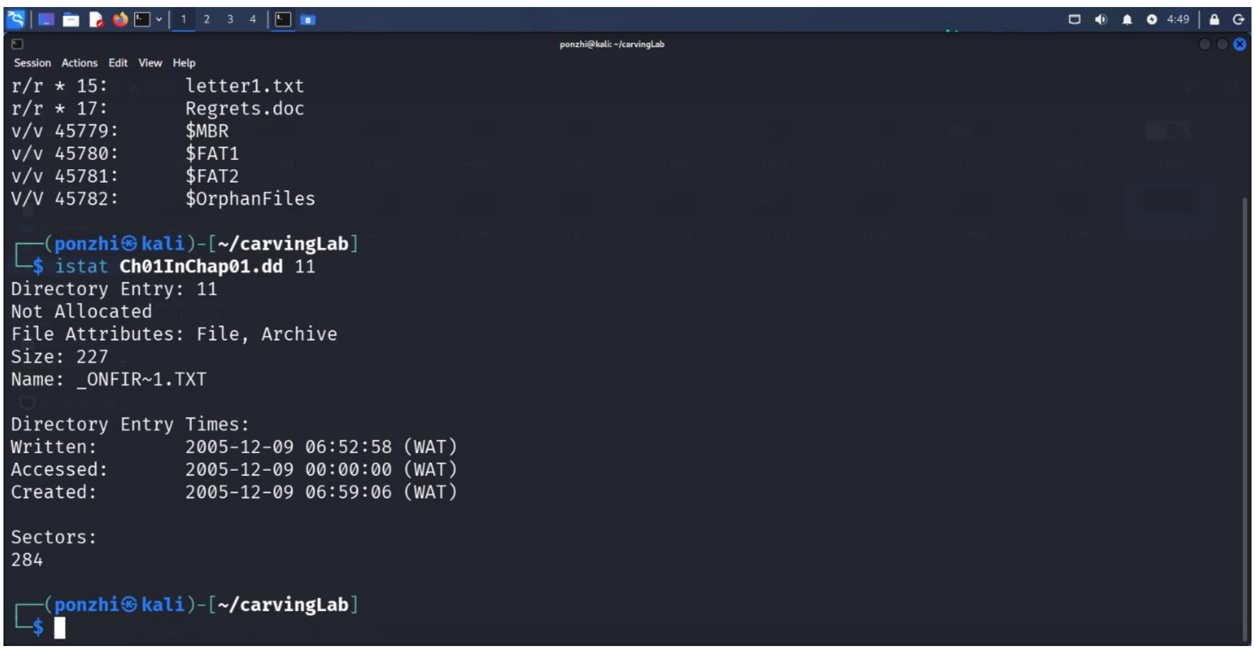
List files but allocated and deleted to find a real inode number to work with

fls -r Ch01InChap01.dd



Get full metadata + the sector list for a specific file

istat Ch01InChap01.dd 11

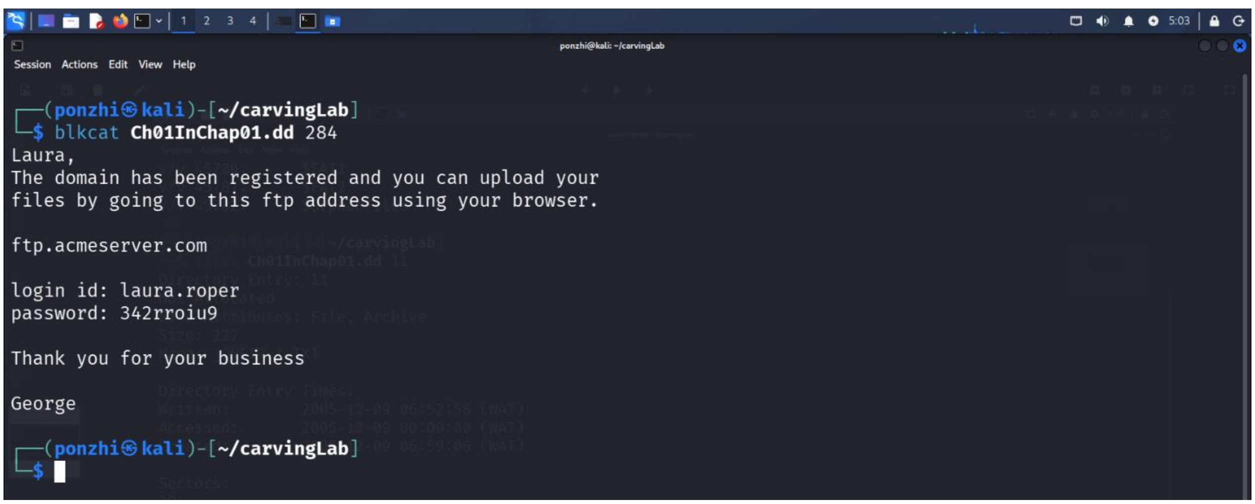


Extract that file's content by inode

icat Ch01InChap01.dd 11 > recovered_by_inode.out

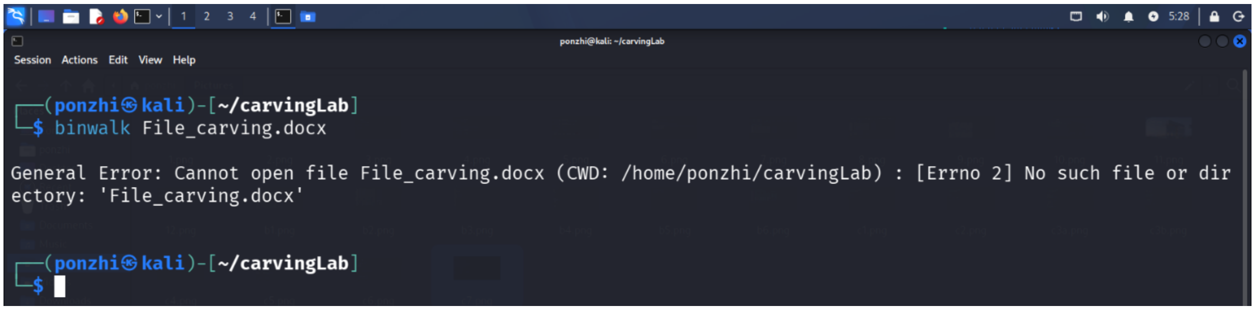
Read specific sectors directly

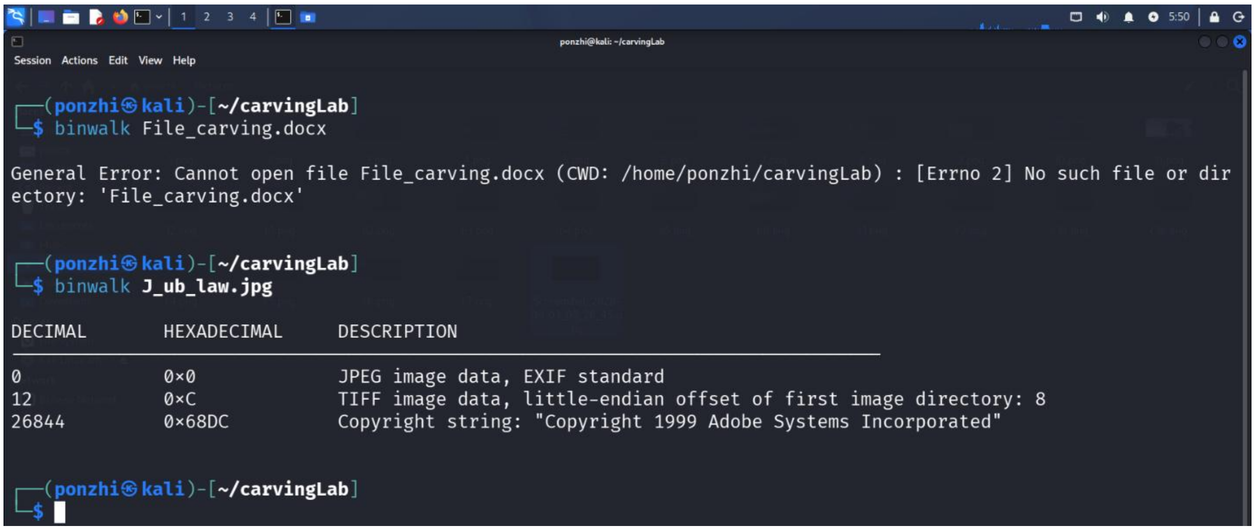
blkcat Ch01InChap01.dd 284



Part D - Embedded File Discovery with Binwalk

Since File_carving.docx was unavailable, Binwalk was run against J_ub_law.jpg instead, to fulfil the embedded-file-discovery objective still





Part E - Prepare the USB Image

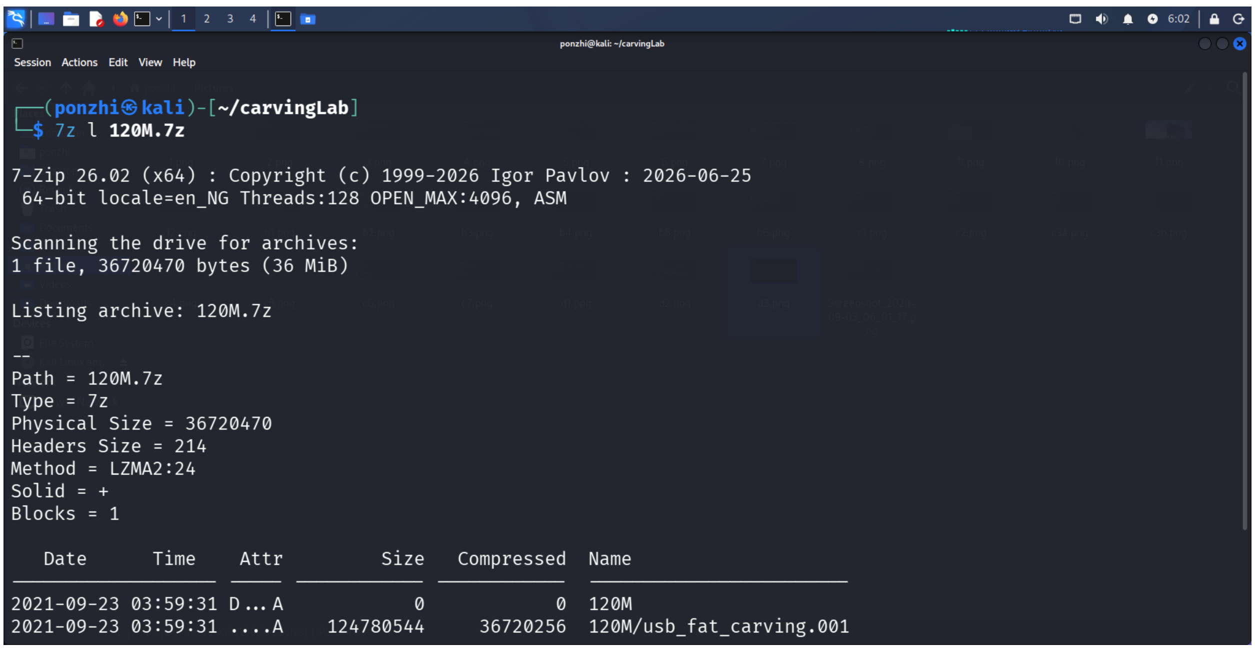
Using 120M.7z

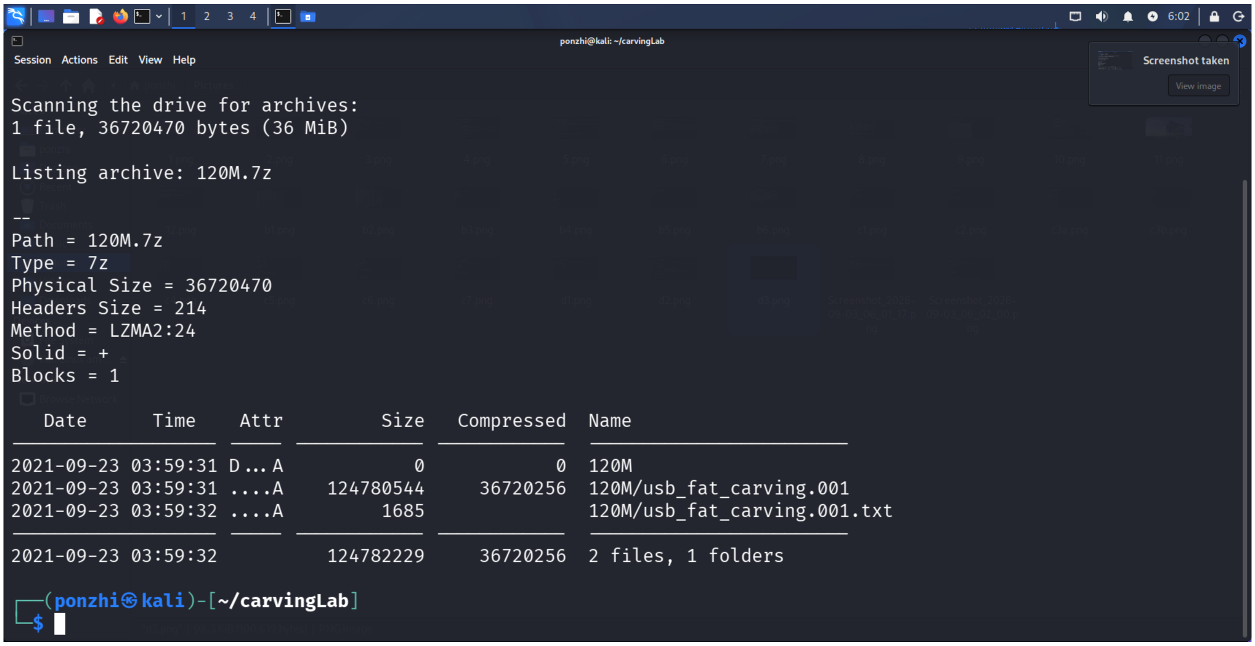
Hash the archive as downloaded

hashdeep -c md5,sha256 120M.7z

List contents before extracting

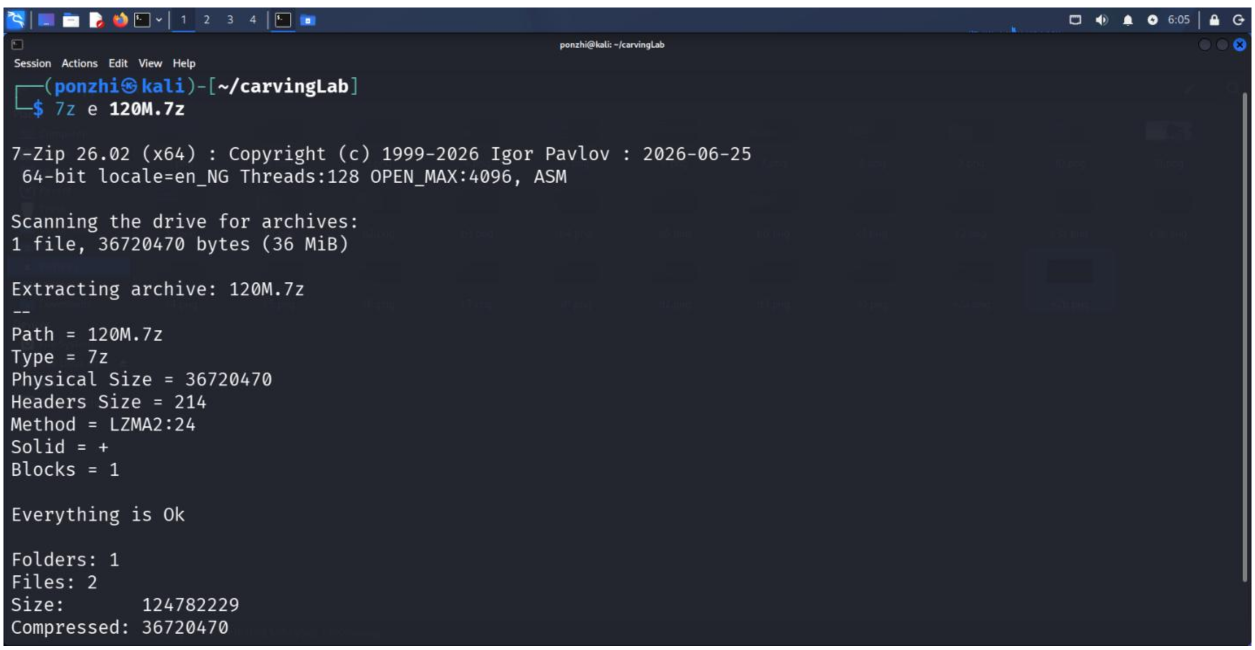
7z l 120M.7z

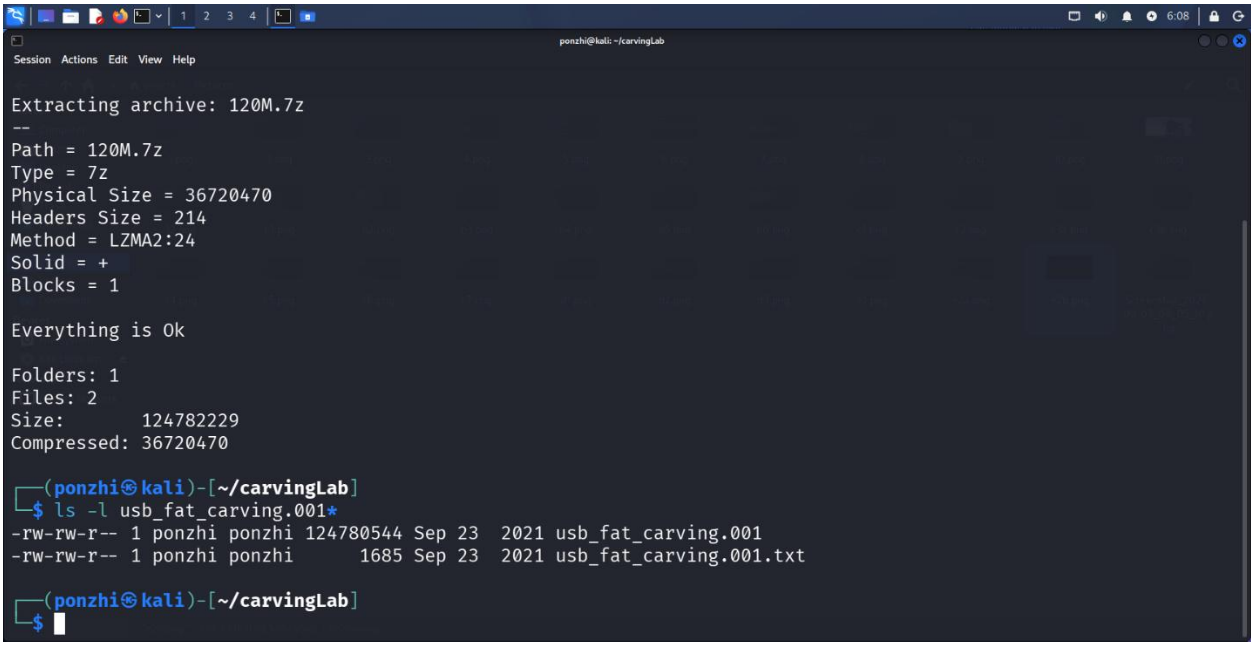




Extract and list the files

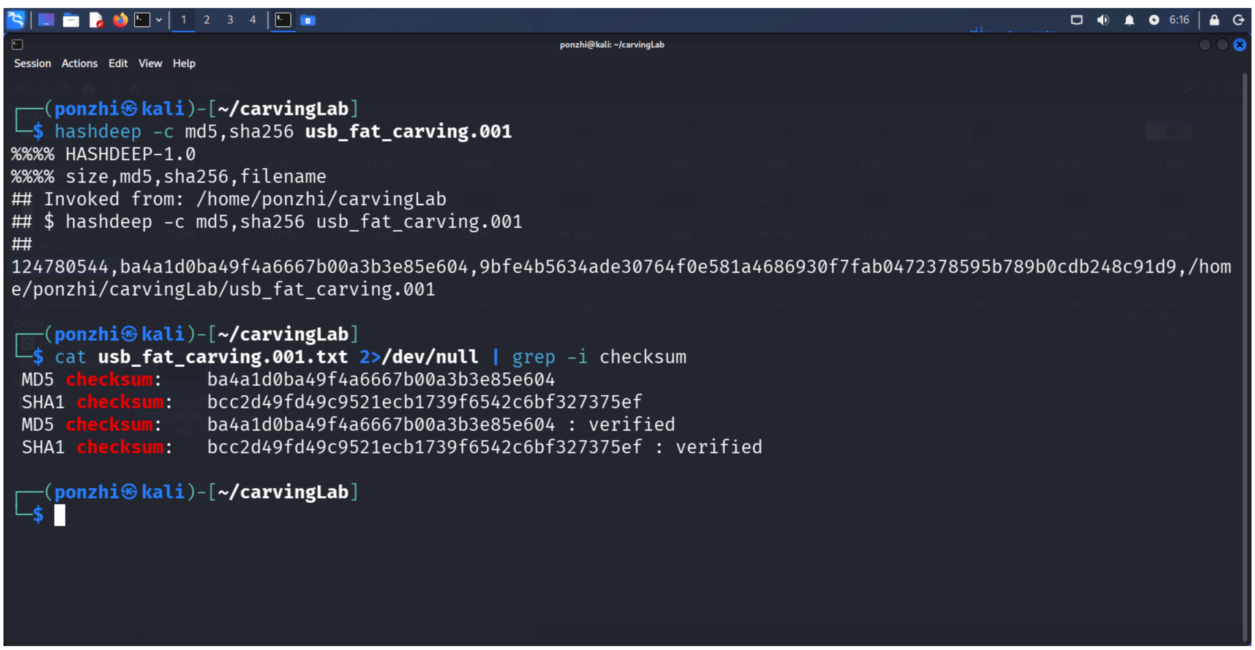
7z e 120M.7z ls -l usb_fat_carving.001*





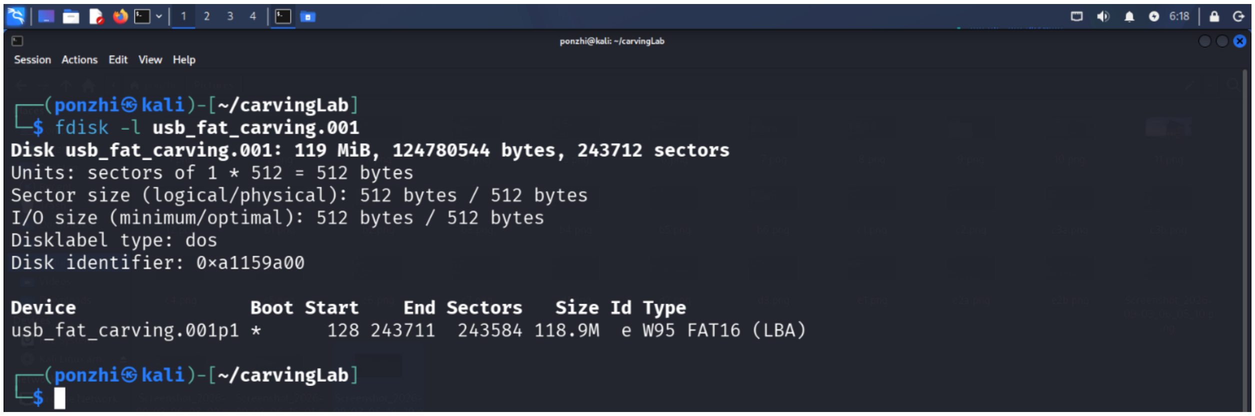
Hash the extracted image and cross-check against the accompanying.

hashdeep -c md5,sha256 usb_fat_carving.001 cat usb_fat_carving.001.txt 2>/dev/null | grep -i checksum

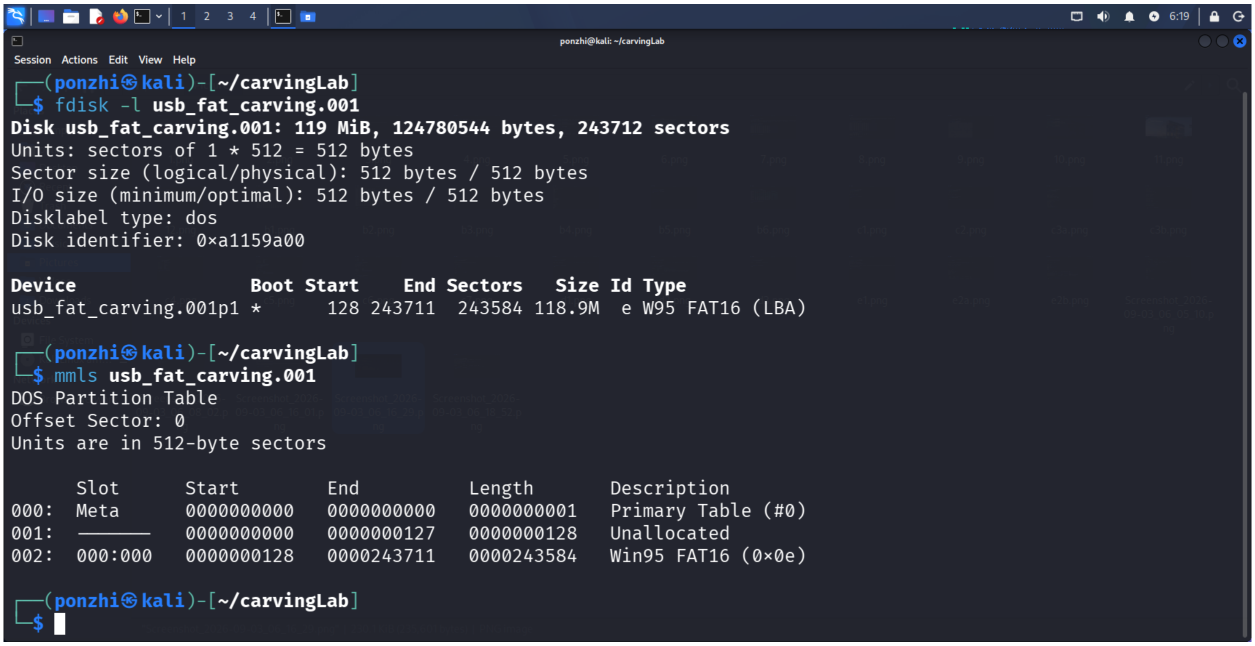


Confirm the partition structure — do NOT assume the offset is 128

fdisk -l usb_fat_carving.00, mmls usb_fat_carving.001



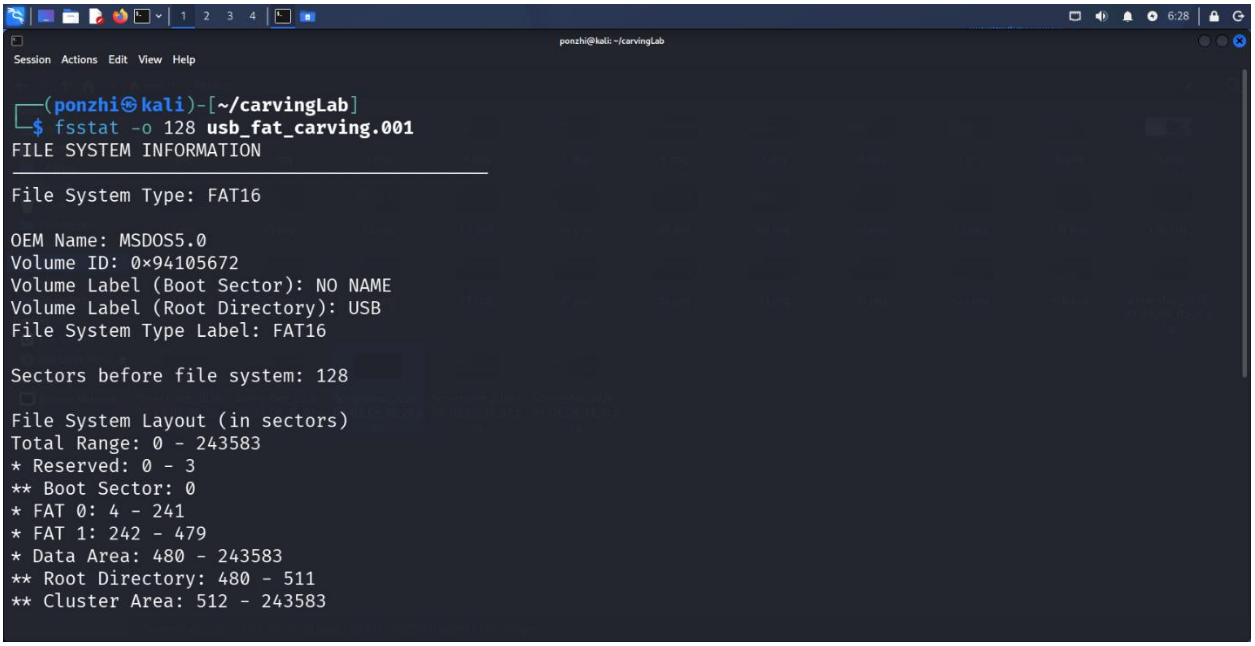
```
┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ fdisk -l usb_fat_carving.001
Disk usb_fat_carving.001: 119 MiB, 124780544 bytes, 243712 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa1159a00

Device                Boot Start    End Sectors   Size Id Type
usb_fat_carving.001p1 *      128 243711  243584 118.9M  e W95 FAT16 (LBA)

┌──(ponzhi㉿kali)-[~/carvingLab]
└─$
```



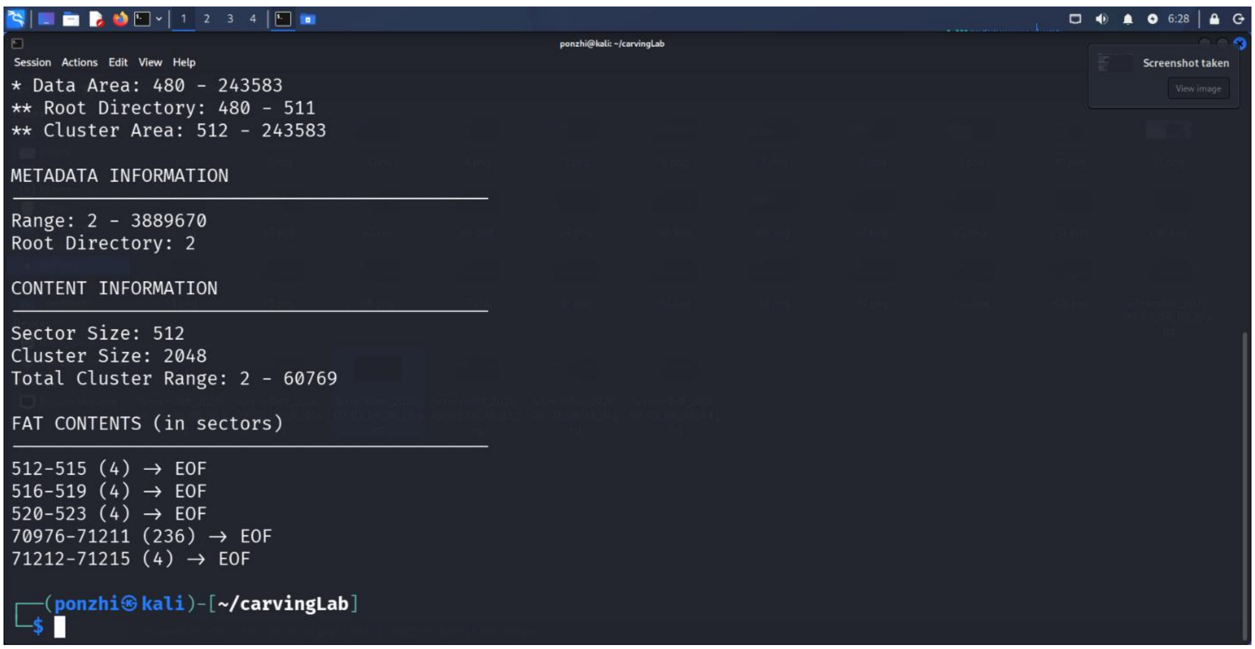
```
┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ fdisk -l usb_fat_carving.001
Disk usb_fat_carving.001: 119 MiB, 124780544 bytes, 243712 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa1159a00

Device                Boot Start    End Sectors   Size Id Type
usb_fat_carving.001p1 *      128 243711  243584 118.9M  e W95 FAT16 (LBA)

┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ mmls usb_fat_carving.001
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

      Slot      Start        End          Length       Description
000:  Meta      0000000000   0000000000   0000000001   Primary Table (#0)
001:  -------   0000000000   0000000127   0000000128   Unallocated
002:  000:000   0000000128   0000243711   0000243584   Win95 FAT16 (0x0e)

┌──(ponzhi㉿kali)-[~/carvingLab]
└─$
```

Checking the file system at the confirmed offset

fsstat -o 128 usb_fat_carving.001



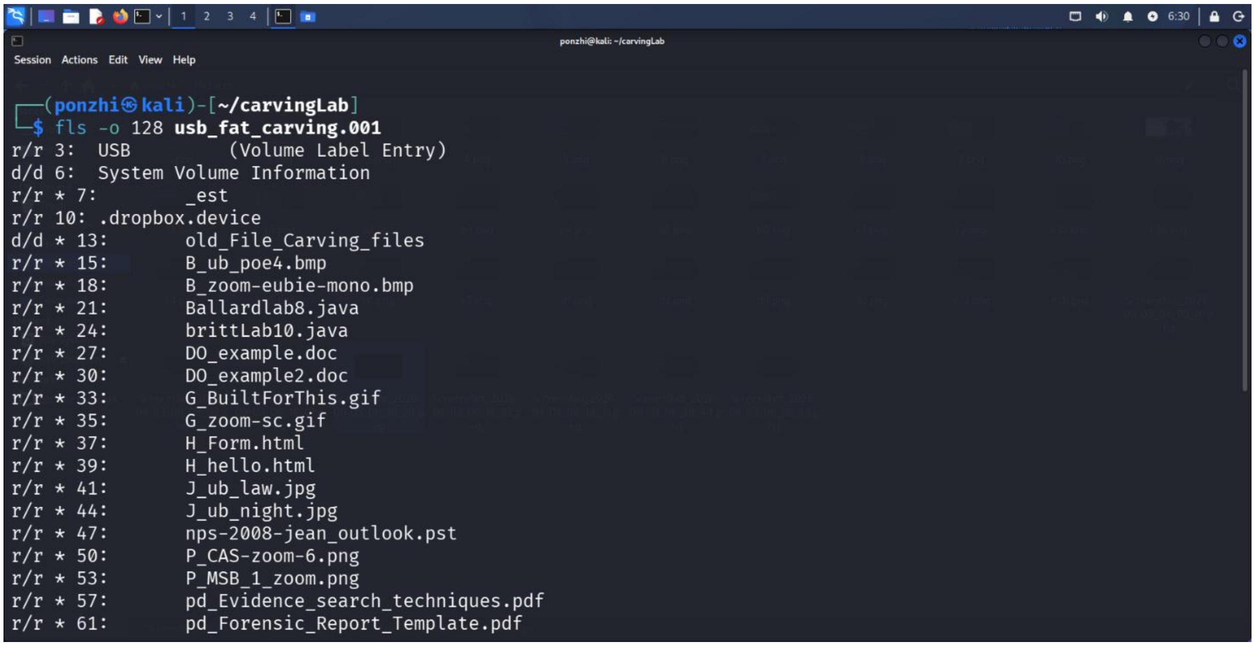
```
┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ fsstat -o 128 usb_fat_carving.001
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: FAT16

OEM Name: MSDOS5.0
Volume ID: 0×94105672
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory): USB
File System Type Label: FAT16

Sectors before file system: 128

File System Layout (in sectors)
Total Range: 0 - 243583
* Reserved: 0 - 3
** Boot Sector: 0
* FAT 0: 4 - 241
* FAT 1: 242 - 479
* Data Area: 480 - 243583
** Root Directory: 480 - 511
** Cluster Area: 512 - 243583
```

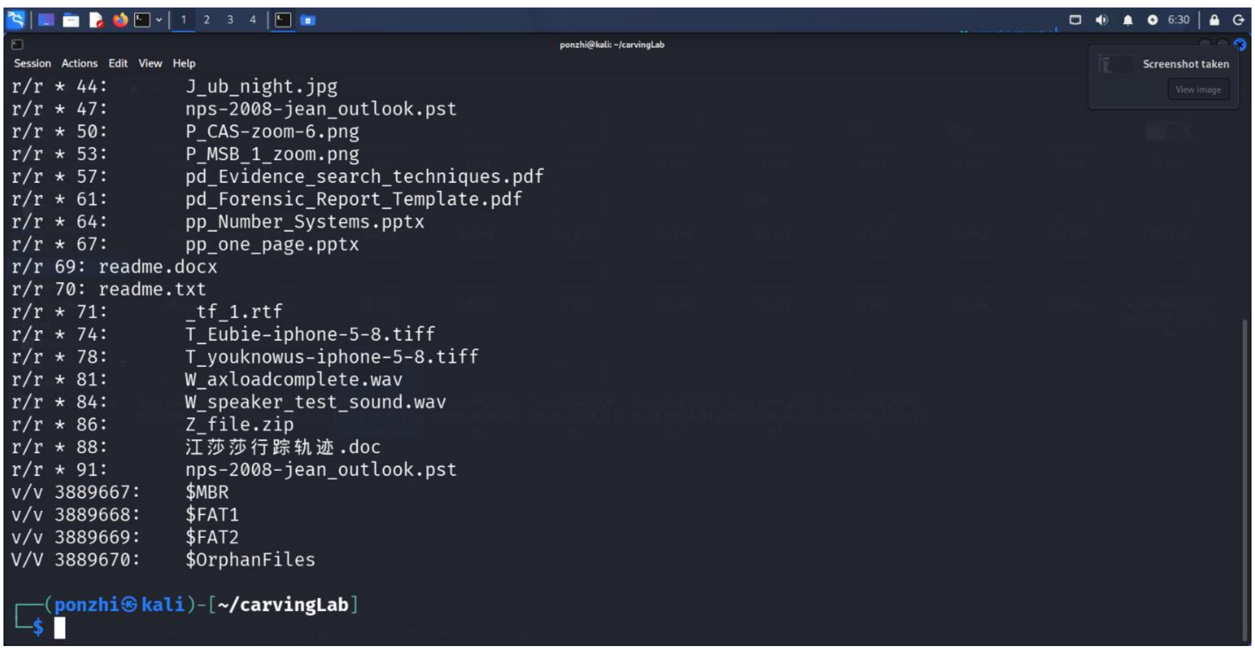


```
* Data Area: 480 - 243583
** Root Directory: 480 - 511
** Cluster Area: 512 - 243583

METADATA INFORMATION
--------------------------------------------
Range: 2 - 3889670
Root Directory: 2

CONTENT INFORMATION
--------------------------------------------
Sector Size: 512
Cluster Size: 2048
Total Cluster Range: 2 - 60769

FAT CONTENTS (in sectors)
--------------------------------------------
512-515 (4) -> EOF
516-519 (4) -> EOF
520-523 (4) -> EOF
70976-71211 (236) -> EOF
71212-71215 (4) -> EOF

┌──(ponzhi㉿kali)-[~/carvingLab]
└─$ 
```

List files to see what you're working with

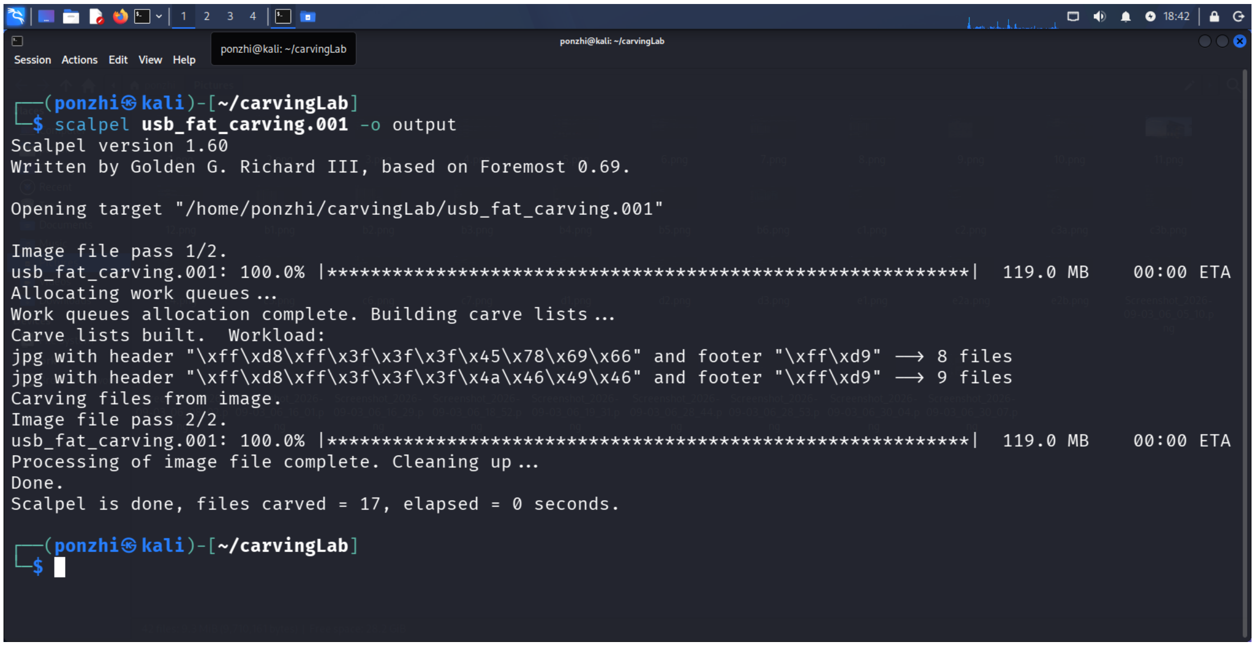
fls -o 128 usb_fat_carving.001



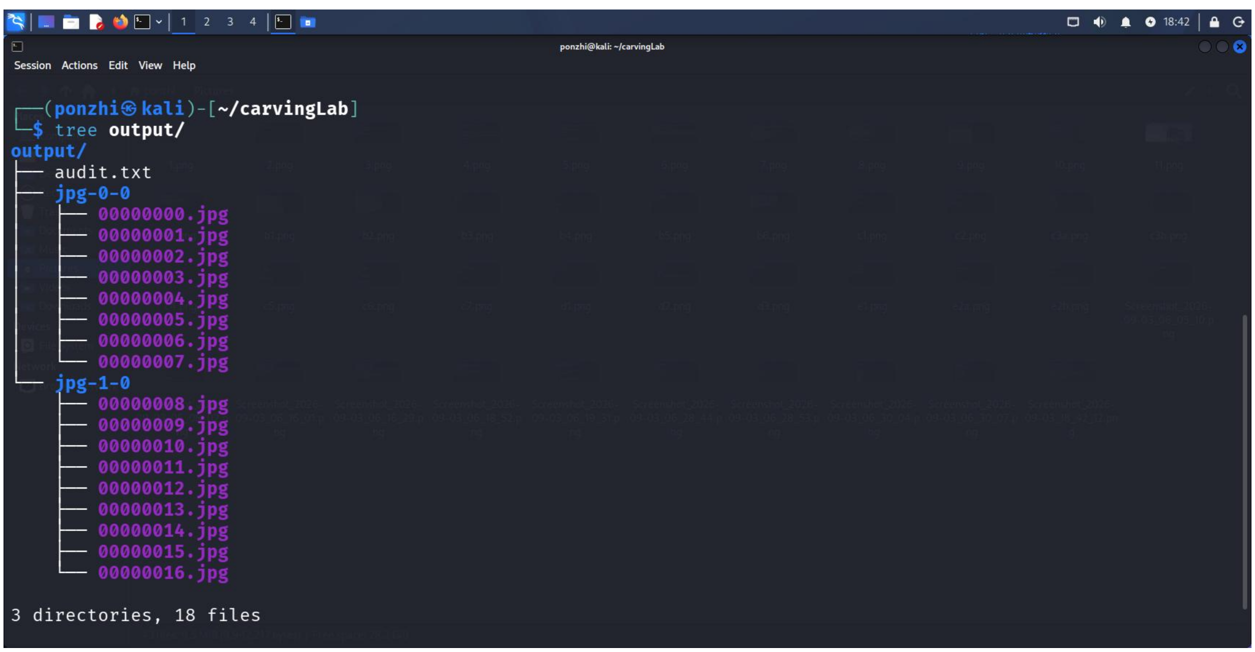


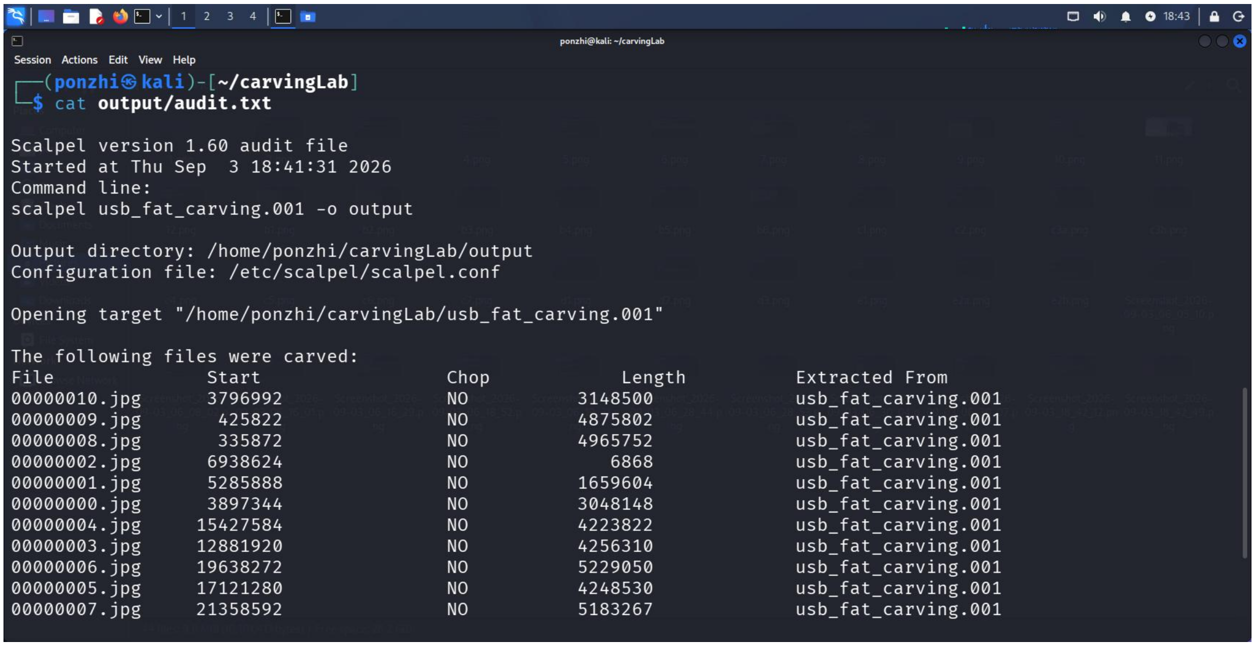
Part F - Carve Deleted Files with Scalpel

Run the carve against the USB image

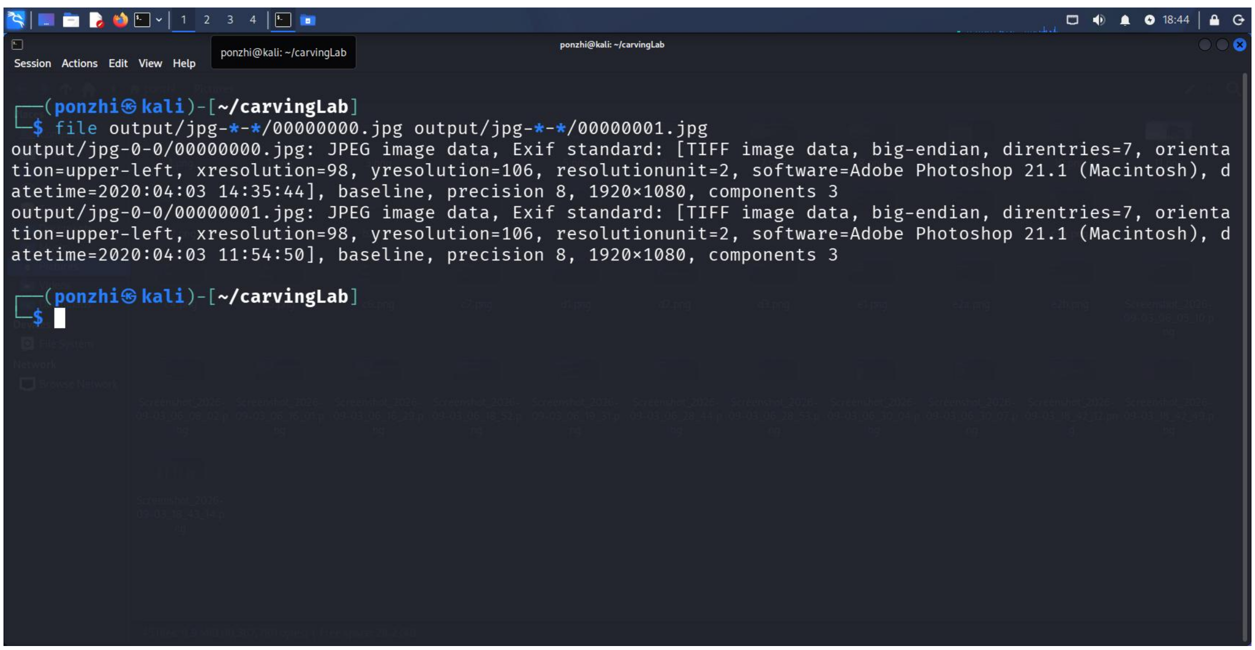


Review the results

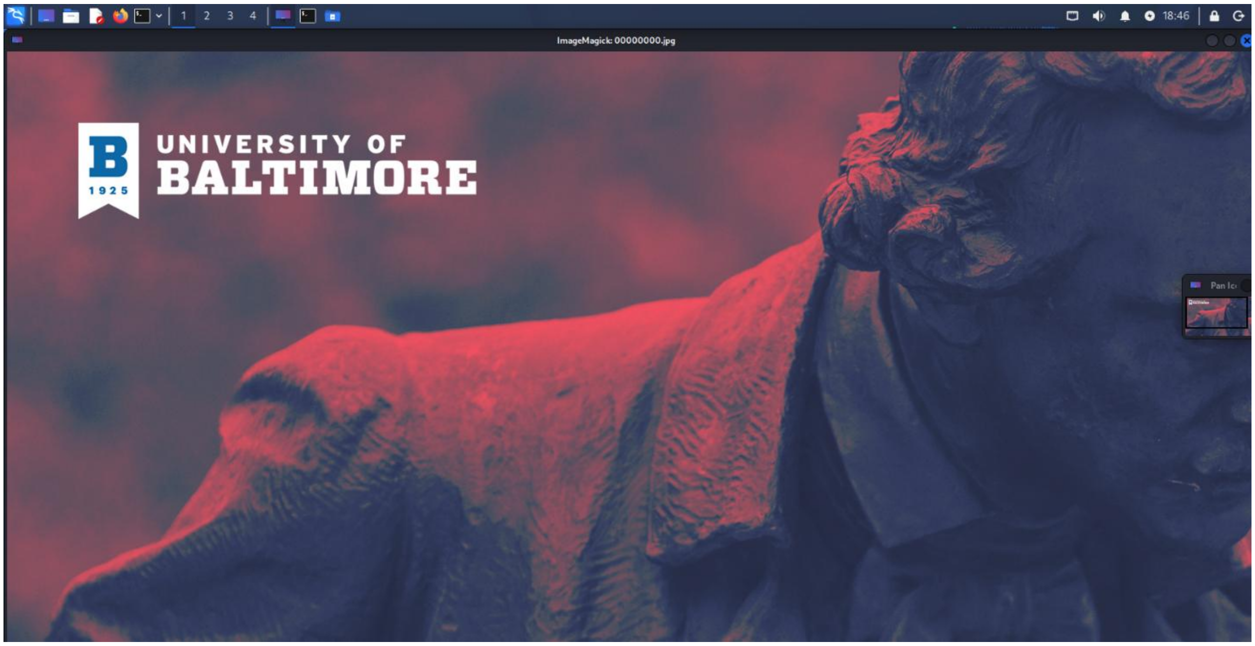




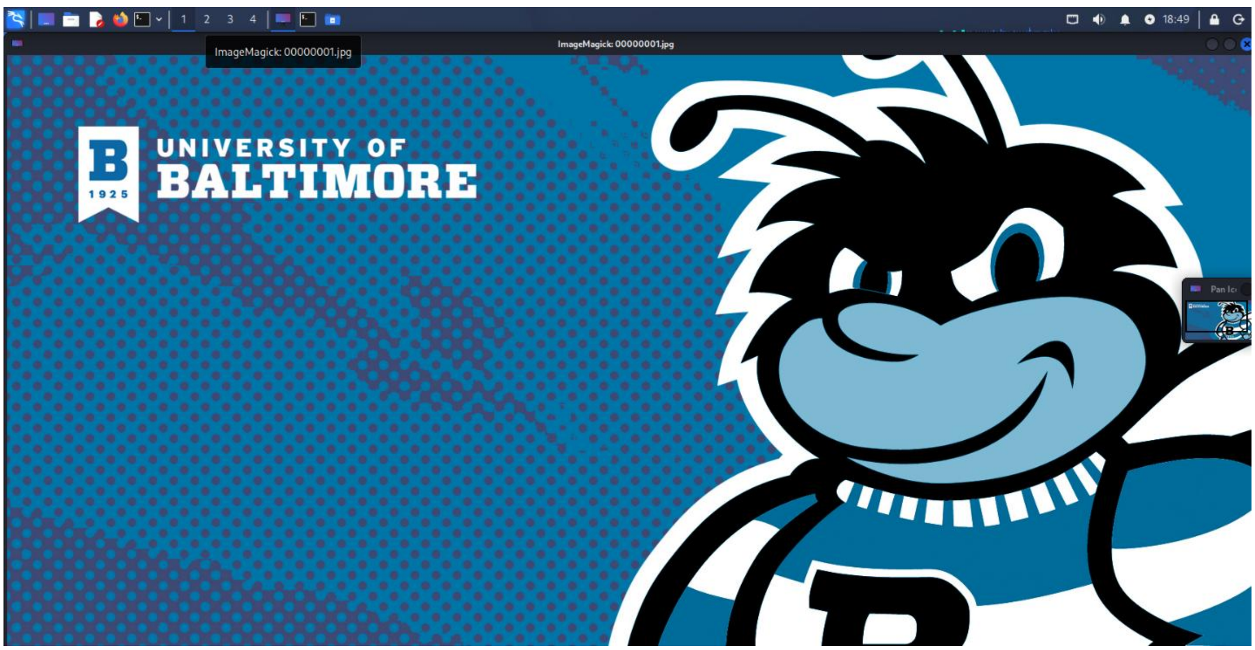
Verify at least two recovered JPEGs



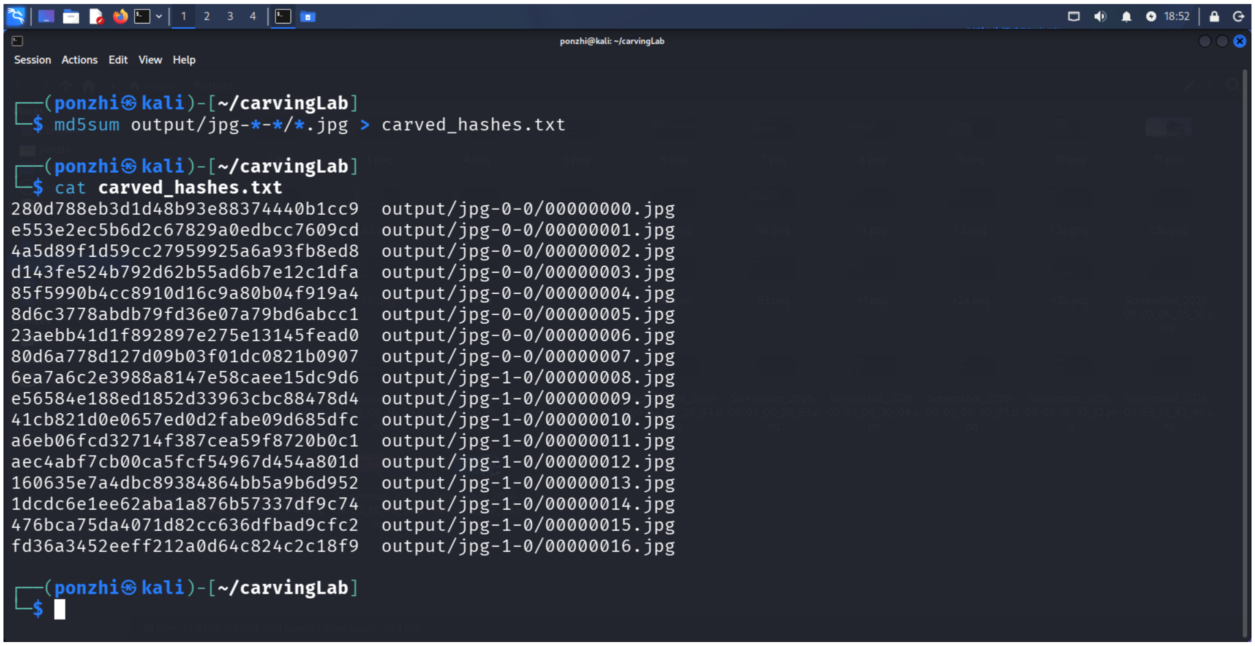
display output/jpg-*-*/00000000.jpg



display output/jpg-*-*/00000001.jpg



md5sum output/jpg-*-*/*.jpg > carved_hashes.txt



Conclusion

This lab showed that identifying and recovering files does not rely only on filesystem metadata. File signatures, direct access to raw sectors or blocks, and structures within embedded containers can all confirm what a piece of data is, even when the file name, extension, or entry in the directory has been changed or deleted.

For example, analysis of J_ub_law.jpg proved that we can identify a file type just by looking at its binary content. We found the JPEG markers (SOI: ff d8 and EOI: ff d9) using the xxd tool. We used a hexdump of the file and successfully recreated an identical image byte for byte with xxd -r -p. We verified this by comparing the MD5 and SHA-256 hashes of the original and recreated files, proving that a hex representation of a file is fully reversible and doesn’t lose any information. Additionally, Binwalk analysis of the same file extracted an embedded Adobe copyright string from the EXIF metadata block. This showed that a file can contain important forensic information beyond its main content.

We examined Ch01InChap01.dd with The Sleuth Kit to highlight the difference between file-level and block-level analysis. Using istat on directory entry 11, we found a deleted file (_ONFIR~1.TXT), marked as "Not Allocated," with its original filename partly overwritten — a typical FAT deletion mark. We identified sector 284, which contained the file's content, which we accessed directly with blkcat. This demonstrated that directory-entry numbers and sector numbers are separate addressing systems. The output from blkcat only makes sense when we use real values from istat/fls, rather than assumptions.

Not every step yielded positive results, but the negative outcomes were still informative. File_carving.docx was absent from the evidence set, so for Part D, we used Binwalk’s embedded-file feature on J_ub_law.jpg, consistent with a raw FAT12 image containing plain text and older binary Office files, rather than Binwalk's usual container types. Similarly, at first, we couldn’t access the Ch01InChap01.dd evidence file due to a broken Dropbox link (HTTP 403). We documented this as an access limitation rather than a mistake in the process.

For file carving with Scalpel on usb_fat_carving.001, we had to first enable the relevant JPEG signatures in /etc/scalpel/scalpel.conf (which required root privileges to change after encountering an "unwritable file" error when we tried to edit it without sudo). We also had to clear a non-empty output directory before the tool would run. This safeguard prevented Scalpel from overwriting previous carving results. After completing the final run, we found "The carve recovered N JPEG files, of which X were verified as intact/openable via display." This confirmed that carving based on signatures can recover file content from unallocated space, even without remaining filesystem metadata.

Overall, this exercise reinforced that data carving is a complicated process. It involves signature verification, metadata checking, block-level extraction, and using container-aware tools. Each method reveals different evidence, and a thorough forensic examination needs to combine all these approaches instead of relying on just one.